EEBus UC Technical Specification

Visualization of Heating Area Name

Version 1.0.0

Cologne, 2022-11-22

EEBus Initiative e.V.
Deutz-Mülheimer-Straße 183
51063 Cologne
GERMANY

Rue d'Arlon 25
1050 Brussels
BELGIUM

Phone: +49 221 / 715 938 - 0
Fax: +49 221 / 715 938 - 99

info@eebus.org
www.eebus.org

District court: Cologne
VR 17275

The EEBus concept was
developed as part
of E-Energy.

Supported by:
Federal Ministry of Economics and Technology
on the basis of a decision by the German Bundestag

Table of contents

List of figures

List of tables

1 Scope of the document

This document describes the Use Case "Visualization of Heating Area Name" (short-name: VHAN). Chapter 2 specifies the High-Level Use Case. Chapter 3 details the technical solution for SPINE for this Use Case. Within this document, a top-down approach is used to derive the requirements for the technical solution from the High-Level description.

1.1 References

1.1.1 EEBUS documents

[ProtocolSpecification]	EEBus_SPINE_TS_ProtocolSpecification.pdf
[ResourceSpecification]	EEBus_SPINE_TS_ResourceSpecification.pdf
[SHIP]	EEBus_SHIP_TS_Specification_v1.0.1.pdf

1.1.2 Normative references

[RFC2119]	IETF RFC 2119: 1997, Key words for use in RFCs to indicate requirement levels Please see section 1.3.1 for details.

1.2 Terms and definitions

Actor
An Actor models a role within a Use Case definition (e.g., an energy manager or an electric vehicle).

HVAC
Abbreviation for Heat, Ventilation and Air Conditioning

Scenario
Part of a Use Case. Splitting a Use Case into Scenarios helps readers understand the Use Case more quickly. Some Scenarios are mandatory for a Use Case, whereas others may be recommended or optional.

Specialization
Reusable data collection for a specific functionality.

SPINE
Smart Premises Interoperable Neutral-message Exchange: Technical Specification of EEBus Initiative e.V.

VHAN
Visualization of Heating Area Name (short name of this Use Case)

1.3 Requirements

1.3.1 Requirements wording

The following keywords are used:

- SHALL
- SHALL NOT
- SHOULD
- SHOULD NOT
- MAY

Note: They apply only if written in capital letters.

For the meaning of the keywords, please refer to [RFC2119].

1.3.2 Mapping of High-Level requirements

Within the High-Level Use Case description, the following abbreviation is used:

[VHAN-xyz]

e.g.: [VHAN-007]

The abbreviation is used to mark High-Level requirements or rules of this Use Case with a unique number xyz. These requirements are referenced throughout the technical solution to show how each High-Level requirement is realized in the technical part.

2 High-Level description

2.1 Introduction

In a house or premises, there are one or more Heating Circuits which may be divided into Heating Zones. Heating Circuits or Heating Zones usually supply several physical rooms. In the following, these rooms will be referred to as HVAC Rooms. An HVAC Room can also be a virtual room if the physical rooms are not represented as independent components in the heating system.

The name of the Heating Circuit(s), Heating Zone(s) and HVAC Room(s) can be requested with this Use Case.

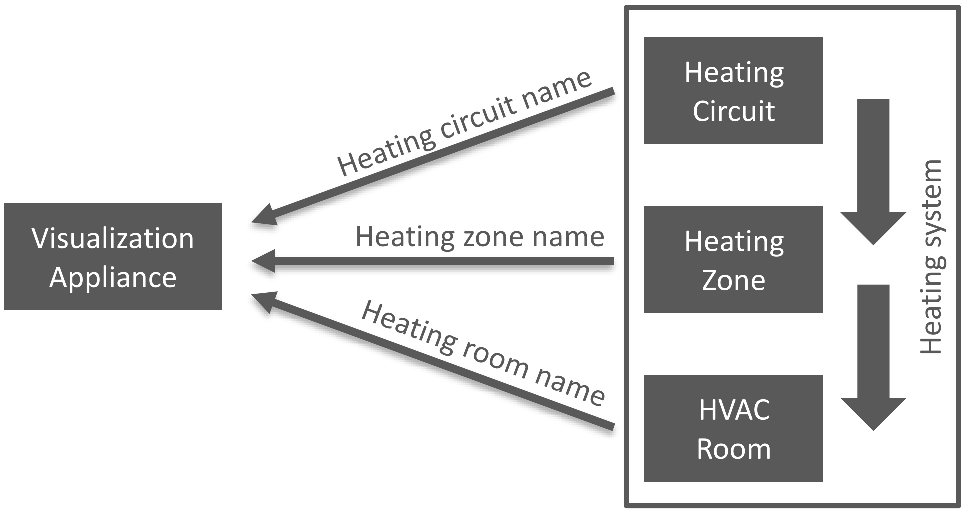

Figure 1: High-Level Use Case functionality overview

Added value: The names can help identify the Heating Circuit(s), Heating Zone(s) and HVAC Room(s) during installation or maintenance.

2.2 Actors

2.2.1 Visualization Appliance

The Actor Visualization Appliance visualizes the names of the Heating Circuit(s), Heating Zone(s) or HVAC Room(s), e.g. to a user or a service employee.

2.2.2 Heating Circuit

A Heating Circuit is connected to the heating system and transports hot water to heat the house or premises.

2.2.3 Heating Zone

A Heating Zone is an optional separation of a Heating Circuit. One Heating Circuit can be separated into several Heating Zones, each with a different flow temperature, if needed.

2.2.4 HVAC Room

The Actor HVAC Room represents a logical or physical room of a house or premises that is considered by the HVAC system.

2.3 Scenarios

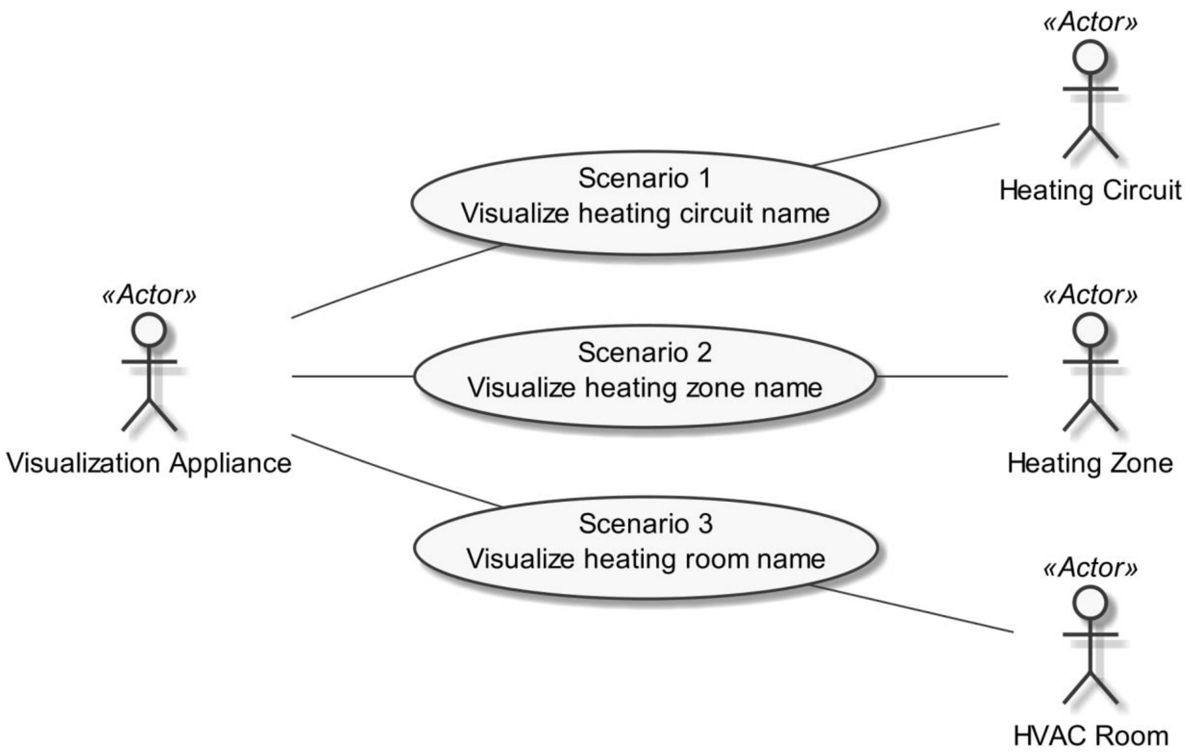

Figure 2: Scenario overview

Scenario number	Scenario name	Visualization Appliance	Heating Circuit	Heating Zone	HVAC Room
1	Visualize heating circuit name	M	M	-	-
2	Visualize heating zone name	M	-	M	-
3	Visualize heating room name	M	-	-	M

Table 1: Scenario implementation requirements for Actors

2.3.1 Scenario 1 - Visualize heating circuit name

2.3.1.1 Description

With this Scenario the name of the respective Actor Heating Circuit can be requested [VHAN-001] and (e.g.) visualized to the user.

2.3.1.2 Conditions

Triggering Event:
The Actor Visualization Appliance is interested in the name of the Actor Heating Circuit.

Pre-condition:
The Actor Visualization Appliance does not know the name of the Actor Heating Circuit.

Post-condition:
The Actor Visualization Appliance knows the name of the Actor Heating Circuit.

2.3.2 Scenario 2 - Visualize heating zone name

2.3.2.1 Description

With this Scenario the name of the respective Actor Heating Zone can be requested [VHAN-002] and (e.g.) visualized to the user.

2.3.2.2 Conditions

Triggering Event:
The Actor Visualization Appliance is interested in the name of the Actor Heating Zone.

Pre-condition:
The Actor Visualization Appliance does not know the name of the Actor Heating Zone.

Post-condition:
The Actor Visualization Appliance knows the name of the Actor Heating Zone.

2.3.3 Scenario 3 - Visualize heating room name

2.3.3.1 Description

With this Scenario, the name of the respective Actor HVAC Room can be requested [VHAN-003] and (e.g.) visualized to the customer or user.

2.3.3.2 Conditions

Triggering Event:
The Actor Visualization Appliance is interested in the name of the Actor HVAC Room.

Pre-condition:

The Actor Visualization Appliance does not know the name of the Actor HVAC Room.

Post-condition:

The Actor Visualization Appliance knows the name of the Actor HVAC Room.

2.4 Dependencies to other Use Cases

The Actor HVAC Room is also used in the following Use Cases:

- "Monitoring of Room Temperature"
- "Configuration of Room Heating Temperature"
- "Configuration of Room Cooling Temperature"
- "Monitoring of Room Cooling System Function"
- "Monitoring of Room Heating System Function"
- "Configuration of Room Cooling System Function"
- "Configuration of Room Heating System Function"

With this Use Case, a name for the HVAC Room as well as its parents within the hierarchical device tree can be provided.

2.5 Assumptions and Prerequisites

None.

3 Technical SPINE solution

3.1 General rules and information

3.1.1 Underlying technology documents

This technical solution relies on the SPINE Resources Specification version 1.1.0 [ResourceSpecification].

For interoperable connectivity this technical solution relies on:

- SPINE Protocol Specification version 1.1.0 [ProtocolSpecification] as application protocol.
- SHIP Specification version 1.0.1 [SHIP] as transport protocol.

3.1.2 Use Case Discovery rules

Use Case discovery SHOULD be supported by each Actor. If Use Case discovery is supported the following rules SHALL apply:

- The string content for the Element "nodeManagementUseCaseData. useCaseInformation. useCaseSupport. useCaseName" within the Use Case discovery (please refer to [ProtocolSpecification]) SHALL be "visualizationOfHeatingAreaName". The string content SHALL only be defined by this Use Case (regardless of the Use Case version).
- The string content of the Element "nodeManagementUseCaseData. useCaseInformation. actor" within the Use Case discovery (please refer to [ProtocolSpecification]) SHALL be set to the according value stated within the corresponding Actor's section.
- An Actor A that is implemented to support this Use Case specification SHALL set the Element "nodeManagementUseCaseData. useCaseInformation. useCaseSupport. useCaseVersion" within the Use Case discovery (please refer to [ProtocolSpecification]) to "1.0.0".
- If an Actor A supports multiple versions of this Use Case with the same major version number, only the highest one SHOULD be set within the Use Case discovery.
- If an Actor A finds a proper counterpart Actor B for this Use Case that supports multiple versions of this Use Case with the same major version number as supported by Actor A, the Actor A SHOULD evaluate from these versions of Actor B only the highest version number.
- If an Actor A supports multiple versions of this Use Case with different major version numbers, for each major version number only the highest version number SHOULD be set within the Use Case discovery.
- If an Actor A finds a proper counterpart Actor B for this Use Case that supports only versions with a major version number not implemented by Actor A, it still might be possible to run the Use Case or parts of the Use Case. Therefore, the Actor A should try to evaluate the Actor B as a valid partner for this Use Case.

3.1.3 Rules for "Content of Specialization..." tables and "Content of Function..." tables

3.1.3.1 General presence indication definitions

Abbreviations for the presence indication of Elements listed in the tables are defined as follows:

Abbreviation	Meaning	Link to requirement keywords
M	Mandatory	SHALL
R	Recommended	SHOULD
O	Optional	MAY

Table 2: Presence indication description

An Actor MAY support Elements that are not listed in the tables. However, another Actor MAY ignore these Elements.

The presence indications "M", "R" and "O" are always meant relative to the respective parent Element. I.e. if a parent Element is optional ("O") and a child is mandatory ("M") the child Element can only be present if the parent Element is present as well.

Note: The indications and the aforementioned rules apply for "complete messages" (so-called "full function exchange", please refer to [ProtocolSpecification]). In contrast, the so-called "restricted function exchange" is designed to permit exchange of specific excerpts of data, i.e. fewer Elements than potentially available from the data owner (partially even not all "mandatory" Elements).

3.1.3.2 Presence indications for "Content of Specialization..." tables

This section only defines rules for the client side.

Elements that are marked with "M" SHALL be supported by the client in case of readable as well as writeable data. This Element may be optional on the server side.

The following applies for readable data that is exchanged in a "read/reply" or "notify" operation:

- "R" means that the data SHOULD be supported by the client. In other words: If the server responds with the according Element, the client SHOULD be able to interpret the according Elements.
- "O" means that the data MAY be supported by the client. In other words: If the server responds with the according Element, the client MAY be able to interpret the according Elements.

The following applies for writeable data that is exchanged in a "write" operation:

- "R" means that the data SHOULD be written by the client.
- "O" means that the data MAY be written by the client.
- "F" means that the data SHALL NOT be written by the client.

The following applies for Elements that are not listed in the Actor section:

- In case of a received "reply" message: The client MAY ignore the Element.
- In case of a "write" operation to be created: The client MAY set the Element but SHALL consider that the server may ignore the Element.
- In case of a received "notify" message: The client MAY ignore the Element.

M, R or O may be combined with the suffix "(event)" to express that a supported Element or value only has to be supported during a certain event and hence does not need to be present at all times. If the event is not active the Element may be omitted or another value may be set. In most cases a High-Level requirement reference for the event is given in the rules column.

In some cases, an Element may have a double presence indication, separated by a colon. Examples:

- M, O
- M \W, O \W

The description (rules) of the Element details in such a case the presence requirement. A typical example is a list-based Element where the "first" (or "last") Element has a different presence requirement than the other list Elements.

3.1.3.3 Presence indications for "Content of Function..." tables

This section only defines rules for the server side.

Elements that are marked with "M" SHALL be supported by the server in case of readable as well as writeable data. In case of writeable data (marked with "M \W") the server does not need to set the Element, because the Element is set only by the client.

The following applies for readable data that is exchanged in a "read/reply" or "notify" operation:

- "R" means that the data SHOULD be provided by the server.
- "O" means that the data MAY be provided by the server.
- "F" means that the data SHALL NOT be provided by the server.

The following applies for writeable data that is exchanged in a "write" operation:

- "R" means that the data SHOULD be supported. In other words: If the client writes the Element, the server SHOULD accept those messages and the contained Elements.
- "O" means that the data MAY be supported. In other words: If the client writes the Element, the server MAY accept those messages and the contained Elements.

The following applies for Elements that are not listed in the Actor section:

- In case of a received "read" request: The according Element MAY be set in the reply.
- In case of a received "write" operation: The server MAY ignore the Element.
- In case of a "notify" operation to be created: The server MAY set the Element.

Note: The server will only accept write operations if the result fulfils the server Function requirements (permitted values, e.g.). Write operations on Elements that are not writeable MAY result in an error message.

M, R or O may be combined with the suffix "(event)" to express that a supported Element or value only has to be supported during a certain event and hence does not need to be present at all times. If the event is not active the Element may be omitted or another value may be set. In most cases a High-Level requirement reference for the event is given in the rules column.

In some cases, an Element may have a double presence indication, separated by a colon. Examples:

- M, O
- M \W, O \W

The description (rules) of the Element details in such a case the presence requirement. A typical example is a list-based Element where the "first" (or "last") Element has a different presence requirement than the other list Elements.

3.1.3.4 Cardinality indications on Elements and list items

A cardinality indication on an Element or list item expresses constraints on the number of occurrences of a given Element or data set. In this section we use "X" as representation for such an Element or data set. Furthermore, "a" and "b" represent constraints. The following rules apply for the occurrence of "X" and its content related to a specific Scenario (see note underneath the list):

1. X
   No cardinality indication.
2. X (a..b)
   This means "X" SHALL occur at least "a" times and at maximum "b" times.
3. X (a..)
   This means "X" SHALL occur at least "a" times and MAY occur more than "a" times.
4. X (..b)
   This means "X" SHALL occur at maximum "b" times and MAY occur less than "b" times (even zero occurrences are permissive).

Note: These rules apply only under consideration of presence indications and with regards to the given Scenario or Function definition for this Use Case.

The following table is an example to explain this for two different placements.

Scenario [{...}]: M/R/O [\W][\C]	**Element**	**Value**	**[High-Level Mapping] Element and value rules**
...	...		...
2: M \W	xFeatureType. xListData. xData. (1..3)		
2: M \W	xId	<*#(1..)>	PRIMARY IDENTIFIER
2: M \W	timePeriod		...
2: M \W	timePeriod. startTime	<xs:duration>	
2: M \W	xSlot. (1..)		
2: M \W	xSlot. xSlotId		...
2: M \W	xSlot. duration	<xs:duration>	...
...	...	...	...

Table 3: Example table for cardinality indications on Elements and list items

The field

xFeatureType. xListData. xData. (1..3)

introduces a data pattern (required Elements and values) for "xData" instances used for Scenario 2. The field itself specifies that such an "xData" instance SHALL occur at least 1 time and at maximum 3 times within "xListData" of Feature Type "xFeatureType". However, this constraint holds only for Scenario 2 and only if such "xData" are required. In this case, they are required, as the left field

2: M \W

denotes that this data set is mandatory for Scenario 2.

The field

xSlot. (1..)

expresses that the Element "xSlot" SHALL occur at least one time within its "xData", but MAY occur more than one time.

For the expression "<*#(1..)>" of Element "xId" please see section 3.1.3.6.

The remaining fields do not have an explicit cardinality indication.

Note: Cardinality expressions are also used within placeholder expressions as defined in section 3.1.3.6. In many cases such placeholder expressions define the number of required or permitted Elements or list items as they explicitly define how many different values for a given Identifier are required or permitted for a given Scenario.

3.1.3.5 Writability and changeability indication

In the same column where the presence indications are denoted, a mark is used to distinguish between writeable, changeable or readable Elements:

- Elements that are marked with "\W" are written by a client and SHALL be writeable at the server according to their presence indications. The client is not obliged to read the according data. Received notifications do not need to be evaluated.
- Elements that are marked with "\C" are changed by a client and SHALL be changeable at the server according to their presence indications. The client is not obliged to read the according data. Received notifications do not need to be evaluated.
- Elements that are marked with "\RW" are read and written by a client and SHALL be writeable and provided by the server according to their presence indications. Received notifications SHALL be evaluated according to their presence indications.
- Elements that are marked with "\RC" are read and changed by a client and SHALL be changeable and provided by the server according to their presence indications. Received notifications SHALL be evaluated according to their presence indications.
- Elements that are not marked are only read by a client and SHALL be provided by the server according to their presence indications. Received notifications SHALL be evaluated according to their presence indications.

"Writeable" means that the Element and its value may be written by a client. This includes the possibility to modify (if the Element is already present), create (if the Element is not present yet), and delete the Element. The server SHALL adjust its Function according to the received "write" operation (unless the server cannot accept the "write" operation according to section 3.1.3.3).

"Changeable" means that the Element's value may be changed by a client. If the Element is not present at the resource before, it probably **cannot** be created by the client via the "write" operation. In this case the server MAY decline such a message.

Note: "\W" includes "\C" already.

Note: Depending on the resource a client might need to request a proper binding before the server accepts a "write" operation.

3.1.3.6 "Value" placeholders

3.1.3.6.1 Introduction

Specializations may use placeholders to model relations between different Elements or even list items of different Functions. The main purpose is to declare which Identifier values relate to each other. As a Use Case does not prescribe specific values to be used for a given Identifier, a placeholder like "<x1>" can be used in "Value" columns to express the intended relations.

There are two styles to identify a placeholder that can be referenced:

1. <xM>
2. <xM#S>

where

1. "x" is any alphabetical prefix like "m", "t", "ec", "cc", etc.
2. "M" is a (major) number like "1", "2", "15", etc.
3. "S" is a sub-number like "1", "7", "10", etc.

Examples for the first style are "<ec1>", "<z12>". Examples for the second style are "<p4#2>", "<m22#3>". For a given placeholder, only one of the styles can be used.

In addition, there are also styles for placeholders that do not need to be referenced:

1. <*>
2. <*#S>

The second style is only used with so-called cardinality expressions.

3.1.3.6.2 Uniqueness of placeholders

A given placeholder <xM> or <xM#S> represents the same value throughout a given Use Case specification for a given set of its parent Identifier values. This shall be explained in a brief example:

We assume a list item with PRIMARY IDENTIFIER "pId". It also has a SUB IDENTIFIER "sId" with placeholder "<s1>". Then, each occurrence of "<s1>" represents the same value for a given value of pId. This means that "<s1>" of a list item with pId=1 can differ from "<s1>" of a list item with pId=2. But it also means that "<s1>" represents the same value in all list items with pId=1.

Note: Typically, parent Identifiers like "pId" will also be expressed with a placeholder like "<p5>", e.g. In this case, the uniqueness rule applies for "<p5>" likewise.

Note: The uniqueness also applies for placeholders used as FOREIGN IDENTIFIER.

3.1.3.6.3 Placeholder expressions with cardinalities

For some Identifiers, more than one placeholder is needed. Several notations are used for this purpose, which make use of cardinality expressions. The general notation is as follows:

1. <xM#(a..b)>

This is equivalent to this explicit definition:

At least a and at maximum b placeholders of this list: <xM#1> <xM#2> ... <xM#b>

This means that the implementation of a given Use Case (or Scenario) requires a minimum of "a" distinct values of the respective Identifier. In total, there can be up to "b" distinct values of the respective Identifier.

Additionally, the following notations may occur:

2. <xM#(a..)>
   This is equivalent to "<xM#(a..b)>" with "b" equal to infinity.
3. <xM#(..b)>
   This is equivalent to "<xM#(a..b)>" with "a" equal to zero.

"<xM#(a..)>" has only a lower bound of "a" distinct values, but no upper bound. "<xM#(..b)>", on the other hand, expresses that the Identifier may not be required at all, but it is permitted to have up to "b" distinct values.

Similarly, the cardinality can be used for placeholders that are not referenced, i.e. <*#(a..b)> etc.

Note: The cardinality does NOT express which of the sub-numbers have to be used! I.e., it does NOT mean that the Identifiers <xM#1> ... <xM#a> are always used and just those with larger sub-numbers (<xM#a+1> ... <xM#b>) are optional. If, for instance, a placeholder expression "<xM#(3..5)>" is given, it may well happen that an implementation makes use of <xM#2>, <xM#4>, and <xM#5> (i.e., it does NOT use <xM#1>, <xM#3>). Which sub-numbers are used usually depends on other parts of a Specialization and their references to required placeholders, which is explained in section 3.1.3.6.4.

3.1.3.6.4 References to placeholders and relations

According to the styles for placeholders that can be referenced, an enumeration value "e" can refer to a particular placeholder:

1. e(-><xM>)
2. e(-><xM#S>)

This denotes that "e" is to be used with "<xM>" or "<xM#S>", resp.

Example: A Specialization contains the Elements "mId" and "phase". "mId" is an Identifier with placeholder definition <m2#(1..3)>. "phase" is a string that permits the values "a", "b", and "c" using this expression:

"a"(-><m2#1>)
"b"(-><m2#2>)
"c"(-><m2#3>)

This expresses that the enumeration value "a" is to be used with the placeholder <m2#1>, "b" is to be used with <m2#2> and "c" with <m2#3>.

Similarly, a placeholder "yN" can refer to a particular placeholder:

3. <yN->xM>
4. <yN->xM#S>
5. <yN#T->xM>
6. <yN#T->xM#S>

where "T" is a sub-number of "yN".

It is also feasible to associate placeholders with cardinalities:

7. <yN#(a..b)->xM#(a..b)>

denotes that <yN#1> is to be used with <xM#1>, <yN#2> is to be used with <xM#2>, etc.

Note: In this case, the placeholder expressions of yN and xM must have the same cardinality.

In some cases, there is a need to express that multiple list items with similar values are feasible or required, but only particular combinations of these different data are permitted. The following example shows that several "fData" list items with different "phase" value are required, but that these list items may only express either the "phase" value combination { "a", "b", "c" } or the "phase" value combination { "a", "abc", "neutral" }. The permitted combinations are defined in a note below a table:

Scenario [{…}]: M/R/O [\W][\C]	**Element**	**Value**	**[High-Level Mapping] Element and value rules**
2: M	F. fListData. fData.		
2: M	zId	<z3#(3..5)>	
2: M	phase	"a"(-><z3#1>)	
		"b"(-><z3#2>)	
		"c"(-><z3#3>)	

		"abc"(-><z3#4>)	
		"neutral"(-><z3#5>)	

Table 4: Content of an example Specialization

Note: One of the following combinations SHALL be used at least: {<z3#1>, <z3#2>, <z3#3>} or {<z3#1>, <z3#4>, <z3#5>}.

3.1.3.7 Rules for content of "Value" column

For a given Scenario, the "Value" column may restrict the permitted content of a Function's Element to one or more particular values. This means that Elements with values deviating from the restriction (i.e. from the permitted values) do not belong to the respective Scenario and need to be considered as if the Element is not set. If more than one particular value is permitted for an Element, the values are in a single line each.

If a presence indication is set for the value (in an additional column before the value), the following rules SHALL be applied:

- "M" means that the value SHALL be supported. This means the value needs to be set at a certain point in time (depending on the value rules) or for a certain Element within a list entry.
- "R" means that the value SHOULD be supported.
- "O" means that the value MAY be supported.

If all possible values of a given mandatory Element are optional or recommended and this Element is used for the purpose of the respective Scenario, one of the values SHALL be set. If all possible values of a given optional or recommended Element are optional or recommended, this Element MAY contain also other values, but then this Element SHALL NOT be considered as part of the respective Scenario.

M, R or O may be combined with the suffix "(event)" to express that a supported value only has to be supported during a certain event and hence does not need to be present at all times. If the event is not active another value may be set. In most cases a High-Level requirement reference for the event is given in the rules column.

If no presence indication is set for the value, the following rules SHALL be applied:

- In case of Elements where the server may set or change an Element on its own (see section 3.1.3.5):
  - within the tables in the "Server data - Resources" sections:
    - the server SHALL support at least one of the listed values.
  - within the tables in the "Client data - Specializations" sections:
    - the client SHALL support all listed values.
- In case of Elements that are writeable or changeable (see section 3.1.3.5):
  - within the tables in the "Server data - Resources" sections:
    - the server SHALL support all listed values.
  - within the tables in the "Client data - Specializations" sections:
    - the client SHALL support at least one of the listed values.

Depending on the Element, different values may be used during runtime. If this is the case, those rules are described within the value rules.

If a value is placed in parenthesis, the corresponding value is a recommendation. The actual value MAY deviate from this, e.g. "(1024)".

3.1.3.8 General information on how to interpret the "Content of Function..." and "Content of Specialization..." tables

Within the "Client data - Specializations" sections each Specialization is described in an own sub-section with the name "Specialization "<name of the Specialization>"" (e.g. "Specialization "Measurement_GridFeedInEnergy""). It contains only one table that includes all Elements needed for this Specialization. The different Functions are mentioned in a continuous row, highlighted with grey background colour. This row contains the following parts:

<Feature Type>. <Function>.[<list entry instance name>.]

The <list entry instance name> is only included if the <Function> is a list-based Function. An example could be:

DeviceConfiguration. deviceConfigurationKeyValueDescriptionListData. deviceConfigurationKeyValueDescriptionData.

In the following rows, only the names of the Elements are stated, without the prefix described above.

Within the "Server data - Resources" sections each Feature Type is described in an own sub-section with the name "Feature Type "<name of the Feature Type>"" (e.g. "Feature Type "Measurement""). It contains sub-sections for each Function named "Function "<name of the Function>"" (e.g. "Function "measurementListData""). These sections contain one table with all Elements needed for this resource. The list entries are mentioned in a continuous row, highlighted with grey background colour. This row contains the following parts:

<Feature Type>. <Function>.[<list entry instance name>.]

The <list entry instance name> is only included if the <Function> is a list-based Function. An example could be:

Measurement. measurementDescriptionListData. measurementDescriptionData.

In the following rows, only the names of the Elements are stated, without the prefix described above.

For both kinds of tables, the following applies:

- Parent Elements are marked with a dot at the end of the name:
  <parent Element>.
  E.g.:
  value.

- If there are sub-Elements, they are described in own rows with the name of the parent Element as prefix, separated by a dot and a blank space:

  <parent Element>. <sub-Element>

  E.g.:

  value. number

3.1.4 Rules for "Feature Types and Functions..." tables

3.1.4.1 Presence indications for "Feature Types and Functions..." tables

The following presence indications are used:

Abbreviation	Meaning	Link to requirement keywords
M	Mandatory	SHALL
R	Recommended	SHOULD
O	Optional	MAY

Table 5: Presence indication of Feature Types and Functions support

If at least one Function of a Feature has the presence indication "M", it is mandatory to support the Feature.

3.1.4.2 Rules for "Possible operations" column

Within the "Feature Types and Functions..." tables the column "Possible operations" state whether the Function is read- or writeable (as defined in the detailed discovery mechanism, see [ProtocolSpecification]).

If the "partial" concept (also called "restricted function exchange") SHALL be supported, the following notation is used (separated for read and write access):

read (M). partial (M)
write (M). partial (M)

If the "partial" concept SHOULD be supported, the following notation is used:

read (M). partial (R)
write (M). partial (R)

If the "partial" concept MAY be supported, the following notation is used:

read (M). partial (O)
write (M). partial (O)

The server can decide whether a notification is submitted complete or partial (as described in [ProtocolSpecification]) if not defined differently within this Use Case Specification.

3.1.5 "Actor ... overview" diagram rules

Within the "Actor [...] overview" diagrams in the "Actors" sub-sections the complete functionality of this Use Case is provided, including optional Scenarios. Which Scenarios are optional can be found in Table 1. The Actor MAY have more functionality implemented than needed for this Use Case.

For the following Actor overview example, a brief description of the graphical symbols will be described.

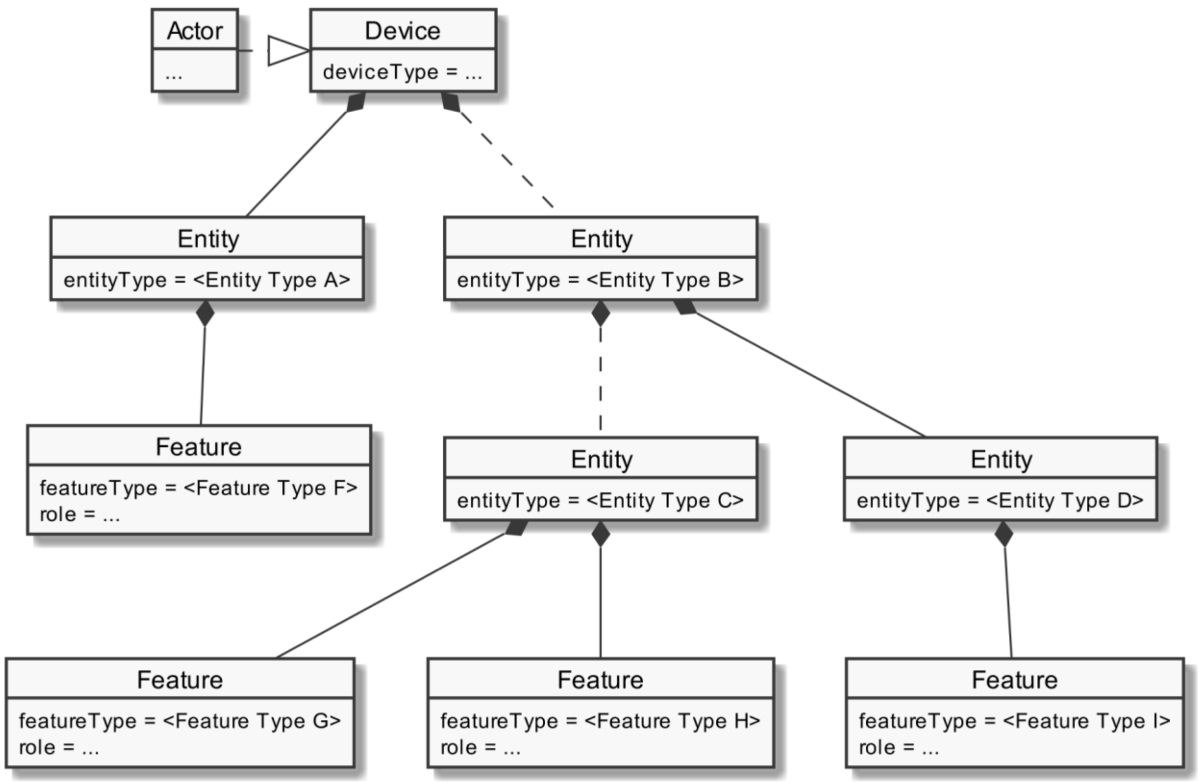

Figure 3: Actor overview example

The solid lines in the figure represent an immediate parent-childhood relation: The Entity with "<Entity Type A>" is a direct child of "Device". The Entity with "<Entity Type D>" is a direct child of the Entity with "<Entity Type B>". All Features are immediate child of the respective Entity.

The dashed lines in the figure express that there MAY be additional Entities between the shown Entities: A vendor's implementation MAY have one or more Entities between "Device" and the Entity with "<Entity Type B>". Likewise, a vendor's implementation MAY have one or more Entities between the Entity with "<Entity Type B>" and the Entity with "<Entity Type C>".

3.1.6 Specializations

Within the "Actors" sub-sections, Specializations are referenced. A Specialization describes a dataset necessary to fulfil the specific requirements of a High-Level Use Case and its Scenarios. Often, data from multiple different Features and Functions are needed to fulfil the requirements. Therefore, a Specialization defines a dataset that may encompass multiple related Functions from one or more different Features.

As different Use Cases sometimes share similar requirements, Specializations are also important from a re-usability perspective. This approach is used to improve consistency across Use Cases and avoid multiple variances of basically the same dataset. This is especially important in the case when an implementation supports multiple Use Cases. E.g. if a power measurement is necessary in two different Use Cases, both Use Cases could define slightly different datasets. In this case, the server as well as the client functionality would have to implement both variances if both Use Cases are supported. This means, depending on the number of Use Cases, two or more datasets need to be generated, transmitted and stored instead of one. Therefore, common Specializations are defined to enable consolidated implementations. Specializations are detailed per Actor, as distinct Actors may use different parts of a Specialization or even have different permissions to change data. In general, this may also depend on the respective Scenario.

If a Feature server can provide the data of a Specialization, the data does not necessarily always need to be available at the Feature server. There might be situations where the user deactivates a Use Case. There may also be other reasons why Use Case data cannot be provided currently. Therefore, a client always needs to be subscribed (as described in section 3.3.4) on the corresponding dataset to stay updated.

The SPINE resource descriptions given in the "SPINE resources of the Actor" sections are derived from the Specializations given in the Actor's overview diagram.

3.1.7 Order of messages within the sequence diagrams

There are several sequence diagrams in this document describing message flows. The order of the messages SHOULD be kept by the communications partners, but there might be cases where a different order makes sense. The communications partners SHALL be able to handle the Scenario functionalities even if the messages are transmitted in a different order by the other Actor(s). The sequence diagrams can be seen as examples.

3.1.8 Further information and rules

None.

3.2 Actors

3.2.1 Visualization Appliance

3.2.1.1 Resource hierarchy

If Use Case discovery is supported (see section 3.1.2) this Actor SHALL be denoted as "VisualizationAppliance" in the Element "nodeManagementUseCaseData. useCaseInformation. actor".

The following diagram provides an overview of the Actor Visualization Appliance's resource hierarchy.

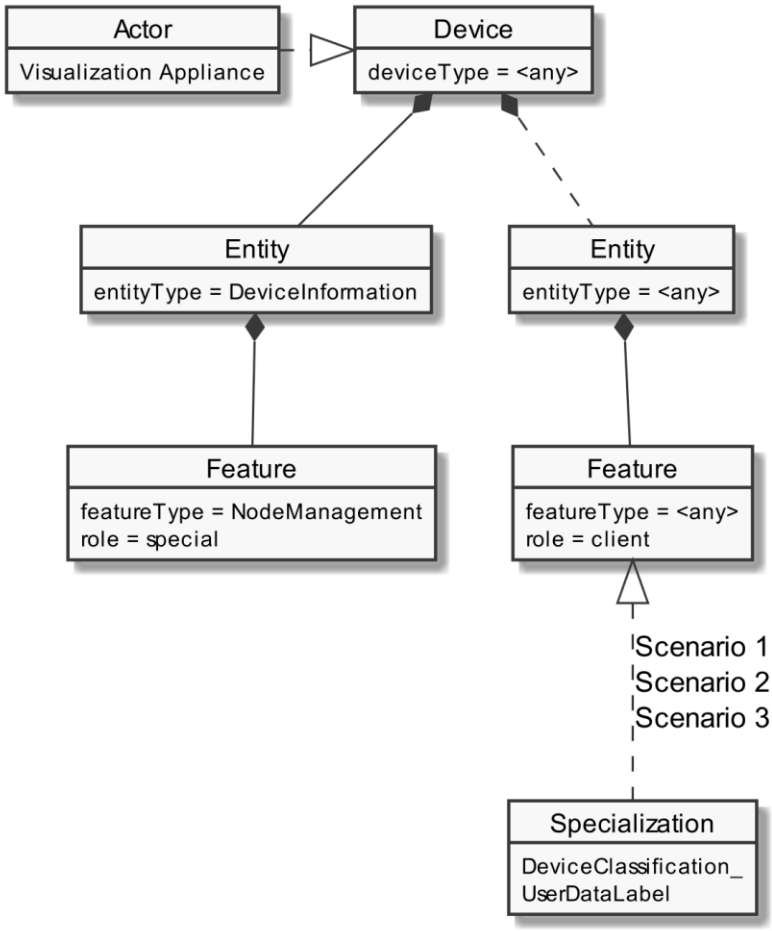

Figure 4: Actor "Visualization Appliance" overview

The ""Actor ... overview" diagram rules" section describes how to interpret the diagram above. See the "Specializations" section for more information regarding the Specializations given in the diagram above.

Note: The entityType "DeviceInformation" with the featureType "NodeManagement" is required by the SPINE protocol and therefore SHALL be supported. Both types are added in the figure for completeness but are not directly linked to the Use Case.

The Use Case specific data follow behind any entityType. The Specializations represent the Scenario specific data that must be supported for each Scenario and are realized through the corresponding featureTypes.

If a Specialization is connected to a Feature with the role "client", the Actor has a client role for this data. This means that the Actor accesses the data set described by the Specialization at a corresponding server Feature. Further details are described in the sub-section "Client data - Specializations".

If a Specialization is connected to a Feature with the role "server", the Actor has the server role for this data. This means that the Actor must provide the corresponding data set of the Specialization as part of its Features. Further details are described in the sub-section "Server data - Resources".

3.2.1.2 Server data - Resources

As this Actor has only client functionality, no resources are described within this section.

3.2.1.3 Client data - Specializations

3.2.1.3.1 Topic "DeviceClassification"

3.2.1.3.1.1 Specialization "DeviceClassification_UserDataLabel"

Scenario [{…}]: M/R/O [\W][\C]	**Element**	**Value**	**[High-Level Mapping] Element and value rules**
1: M 2: M 3: M	DeviceClassification. deviceClassificationUserData.		
1: M 2: M 3: M	userLabel		1: [VHAN-001] 2: [VHAN-002] 3: [VHAN-003] The string-length SHOULD NOT be longer than 256 characters. If it is longer, the sender SHALL consider the possibility that the receiver will shorten the string to 256 characters.

Table 6: Content of Specialization "DeviceClassification_UserDataLabel" at Actor Visualization Appliance

3.2.2 Heating Circuit

3.2.2.1 Resource hierarchy

If Use Case discovery is supported (see section 3.1.2) this Actor SHALL be denoted as "HeatingCircuit" in the Element "nodeManagementUseCaseData. useCaseInformation. actor".

The following diagram provides an overview of the Actor Heating Circuit's resource hierarchy.

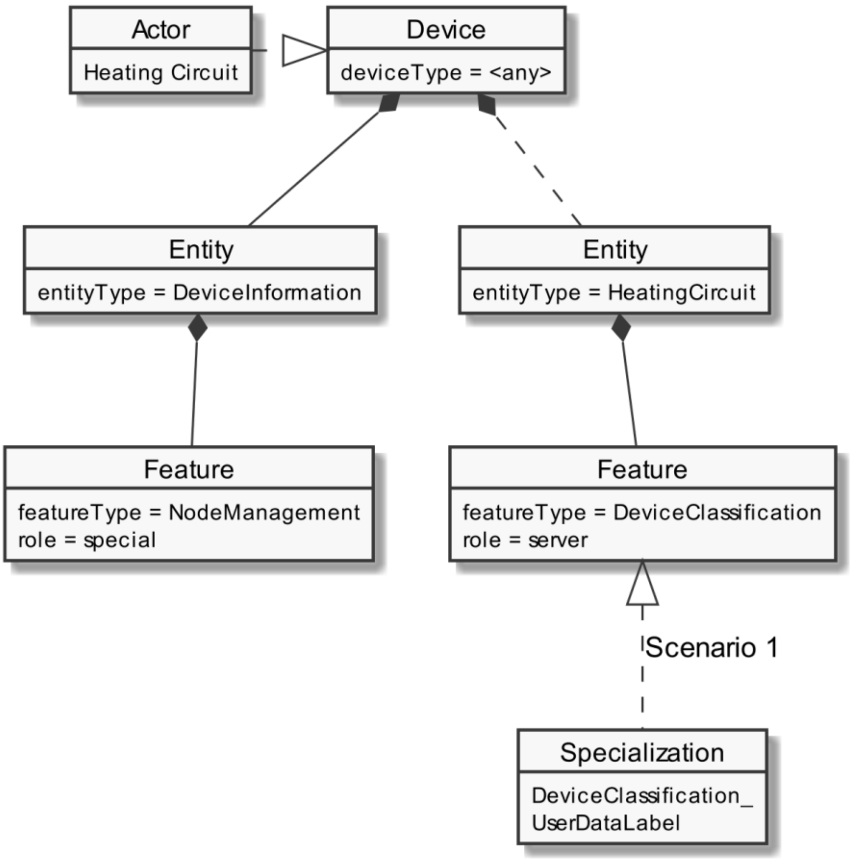

Figure 5: Actor "Heating Circuit" overview

The ""Actor ... overview" diagram rules" section describes how to interpret the diagram above. See the "Specializations" section for more information regarding the Specializations given in the diagram above.

Note: The entityType "DeviceInformation" with the featureType "NodeManagement" is required by the SPINE protocol and therefore SHALL be supported. Both types are added in the figure for completeness but are not directly linked to the Use Case.

The Use Case specific data follow behind the entityType "HeatingCircuit". The Specializations represent the Scenario specific data that must be supported for each Scenario and are realized through the corresponding featureTypes.

If a Specialization is connected to a Feature with the role "client", the Actor has a client role for this data. This means that the Actor accesses the data set described by the Specialization at a corresponding server Feature. Further details are described in the sub-section "Client data - Specializations".

If a Specialization is connected to a Feature with the role "server", the Actor has the server role for this data. This means that the Actor must provide the corresponding data set of the Specialization as part of its Features. Further details are described in the sub-section "Server data - Resources".

3.2.2.2 Server data - Resources

3.2.2.2.1 Overview

Behind the entityType "HeatingCircuit", the Actor Heating Circuit SHALL offer the Feature Types and Functions given in the table below.

Feature Type	Scenario: M/R/O	Function	Possible operations
DeviceClassification	1: M	deviceClassificationUserData	read (M)

Table 7: Feature Types and Functions used within this Use Case by the Actor Heating Circuit

For each of these Feature Types, the following rule applies: There SHALL be at maximum one Feature with the Feature Type in the Entity.

Note: As a consequence of the previous rule, an implementation may need to have Feature data from different Scenarios/Specializations or even Use Cases in a given Feature.

The Scenario number shows in which Scenarios the Heating Circuit acts as server and which Feature Types and Functions are relevant in each Scenario.

A detailed definition of the Elements and values that shall be supported in each Function is given in the following sub-sections.

Note: If in the table above "partial" read is not mentioned or is only optional, it still might be mandatory to support partial notifications. The details of "partial" support are described within the Scenario sections.

Note: The presence indications stated above are meant relative to the ones of the according Scenario stated in Table 1. I.e., if a Scenario is optional ("O") and a Feature Type is mandatory ("M"), the Feature Type need only be supported if the Scenario is supported, too.

Note: Further Features MAY be implemented on the same Entities; also, further Functions MAY be implemented in the used Entities.

3.2.2.2.2 Feature Type "DeviceClassification"

3.2.2.2.2.1 Function "deviceClassificationUserData"

Scenario [{...}]: M/R/O [\W][\C]	Element	Value	[High-Level Mapping] Element and value rules
1: M	DeviceClassification. deviceClassificationUserData.		
1: M	userLabel		[VHAN-001] The string-length SHOULD NOT be longer than 256 characters. If it is longer, the sender SHALL consider the possibility that the

			receiver will shorten the string to 256 characters.

Table 8: Content of Function "deviceClassificationUserData" at Actor Heating Circuit

3.2.2.3 Client data - Specializations

As this Actor has only server functionality, no Specializations are described within this section.

3.2.3 Heating Zone

3.2.3.1 Resource hierarchy

If Use Case discovery is supported (see section 3.1.2) this Actor SHALL be denoted as "HeatingZone" in the Element "nodeManagementUseCaseData. useCaseInformation. actor".

The following diagram provides an overview of the Actor Heating Zone's resource hierarchy.

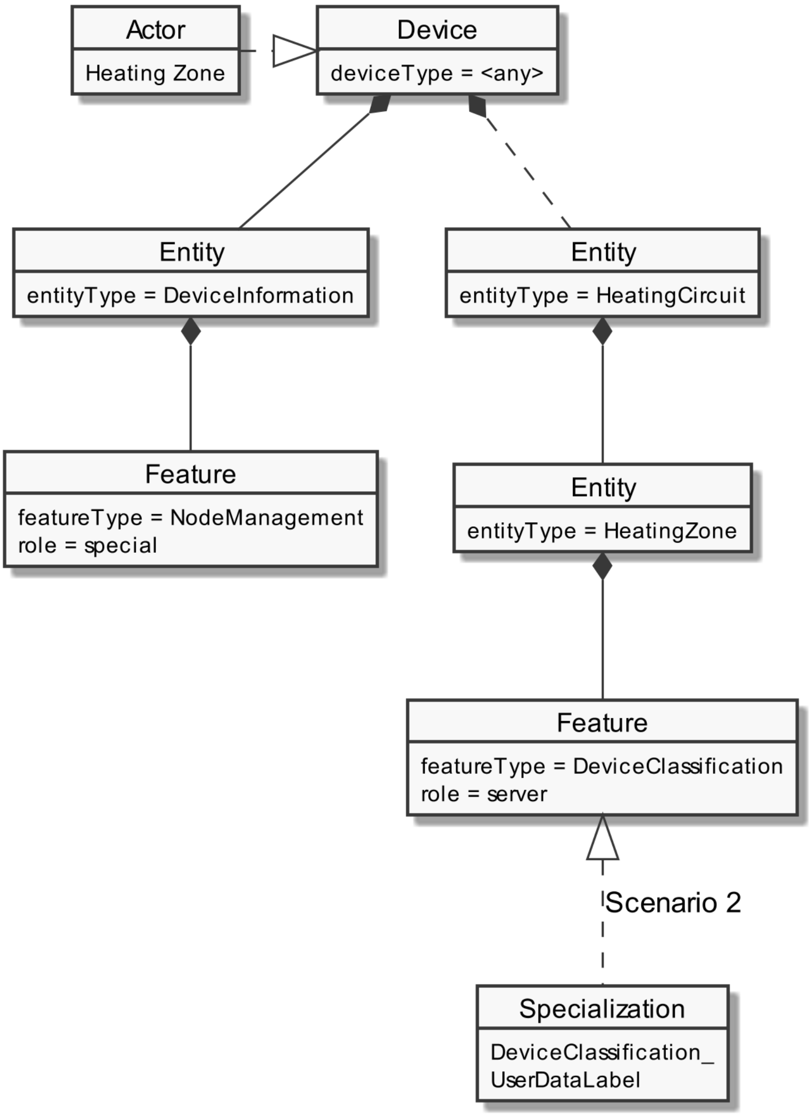

Figure 6: Actor "Heating Zone" overview

The ""Actor ... overview" diagram rules" section describes how to interpret the diagram above. See the "Specializations" section for more information regarding the Specializations given in the diagram above.

Note: The entityType "DeviceInformation" with the featureType "NodeManagement" is required by the SPINE protocol and therefore SHALL be supported. Both types are added in the figure for completeness but are not directly linked to the Use Case.

The Use Case specific data follow behind the entityType "HeatingZone", which is a sub-Entity of the Entity "HeatingCircuit". The Specializations represent the Scenario specific data that must be supported for each Scenario and are realized through the corresponding featureTypes.

If a Specialization is connected to a Feature with the role "client", the Actor has a client role for this data. This means that the Actor accesses the data set described by the Specialization at a corresponding server Feature. Further details are described in the sub-section "Client data - Specializations".

If a Specialization is connected to a Feature with the role "server", the Actor has the server role for this data. This means that the Actor must provide the corresponding data set of the Specialization as part of its Features. Further details are described in the sub-section "Server data - Resources".

3.2.3.2 Server data - Resources

3.2.3.2.1 Overview

Behind the entityType "HeatingZone", the Actor Heating Zone SHALL offer the Feature Types and Functions given in the table below.

Feature Type	**Scenario: M/R/O**	**Function**	**Possible operations**
DeviceClassification	2: M	deviceClassificationUserData	read (M)

Table 9: Feature Types and Functions used within this Use Case by the Actor Heating Zone

For each of these Feature Types, the following rule applies: There SHALL be at maximum one Feature with the Feature Type in the Entity.

Note: As a consequence of the previous rule, an implementation may need to have Feature data from different Scenarios/Specializations or even Use Cases in a given Feature.

The Scenario number shows in which Scenarios the Heating Zone acts as server and which Feature Types and Functions are relevant in each Scenario.

A detailed definition of the Elements and values that shall be supported in each Function is given in the following sub-sections.

Note: If in the table above "partial" read is not mentioned or is only optional, it still might be mandatory to support partial notifications. The details of "partial" support are described within the Scenario sections.

Note: The presence indications stated above are meant relative to the ones of the according Scenario stated in Table 1. I.e., if a Scenario is optional ("O") and a Feature Type is mandatory ("M"), the Feature Type need only be supported if the Scenario is supported, too.

Note: Further Features MAY be implemented on the same Entities; also, further Functions MAY be implemented in the used Entities.

3.2.3.2.2 Feature Type "DeviceClassification"

3.2.3.2.2.1 Function "deviceClassificationUserData"

Scenario [{...}]: M/R/O [\W][\C]	Element	Value	[High-Level Mapping] Element and value rules
2: M	DeviceClassification. deviceClassificationUserData.		
2: M	userLabel		[VHAN-002] The string-length SHOULD NOT be longer than 256 characters. If it is longer, the sender SHALL consider the possibility that the receiver will shorten the string to 256 characters.

Table 10: Content of Function "deviceClassificationUserData" at Actor Heating Zone

3.2.3.3 Client data - Specializations

As this Actor has only server functionality, no Specializations are described within this section.

3.2.4 HVAC Room

3.2.4.1 Resource hierarchy

If Use Case discovery is supported (see section 3.1.2) this Actor SHALL be denoted as "HVACRoom" in the Element "nodeManagementUseCaseData. useCaseInformation. actor".

The following diagram provides an overview of the Actor HVAC Room's resource hierarchy.

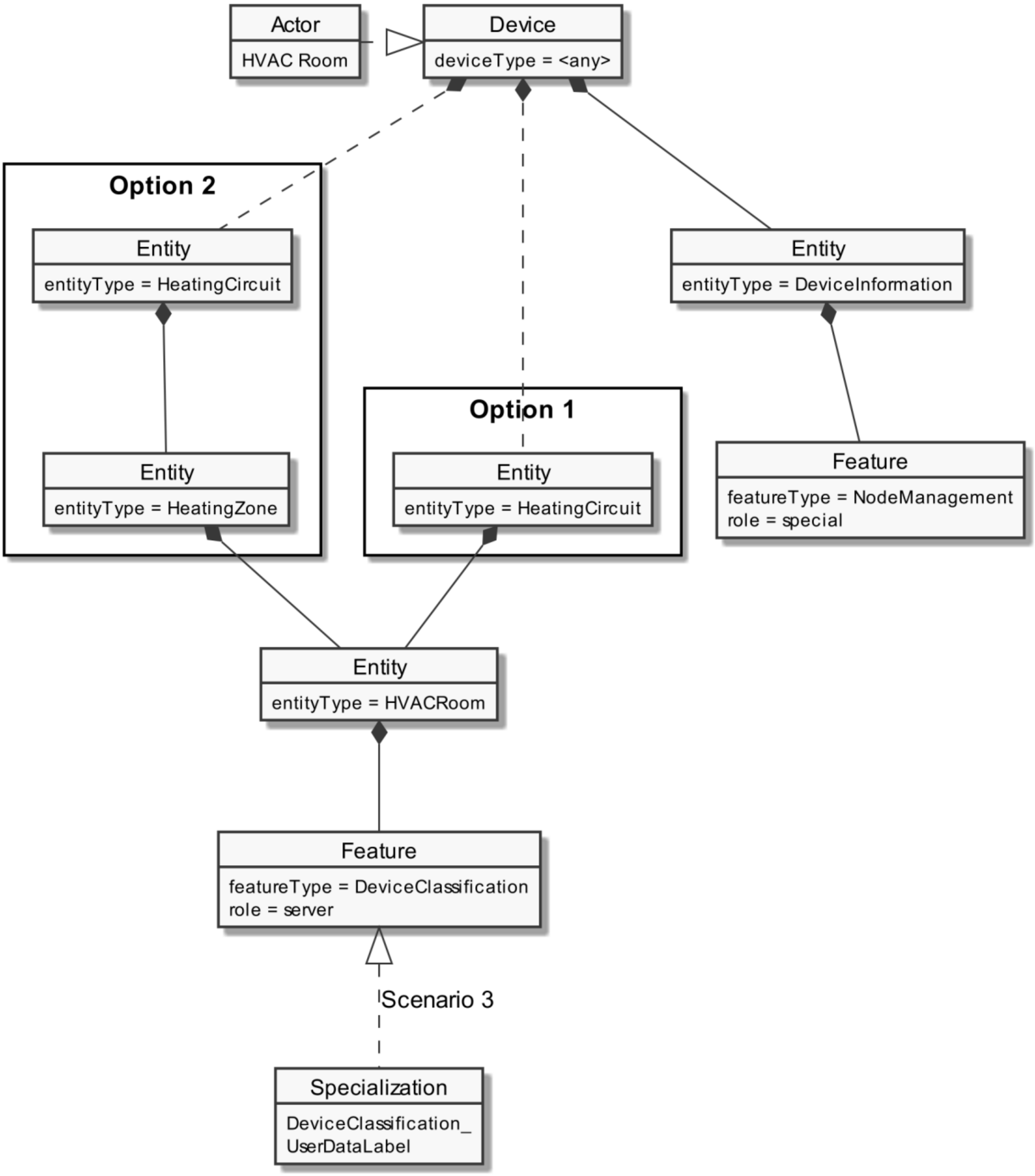

Figure 7: Actor "HVAC Room" overview

The ""Actor ... overview" diagram rules" section describes how to interpret the diagram above. See the "Specializations" section for more information regarding the Specializations given in the diagram above.

Note: The entityType "DeviceInformation" with the featureType "NodeManagement" is required by the SPINE protocol and therefore SHALL be supported. Both types are added in the figure for completeness but are not directly linked to the Use Case.

The Use Case specific data follow behind the entityType "HVACRoom", which is

1. either a sub-Entity of the Entity "HeatingCircuit" (see "Option 1" in the figure),

2. or a sub-Entity of the Entity "HeatingZone" that is a sub-Entity of the Entity "HeatingCircuit" (see "Option 2" in the figure).

The Specializations represent the Scenario specific data that must be supported for each Scenario and are realized through the corresponding featureTypes.

If a Specialization is connected to a Feature with the role "client", the Actor has a client role for this data. This means that the Actor accesses the data set described by the Specialization at a corresponding server Feature. Further details are described in the sub-section "Client data - Specializations".

If a Specialization is connected to a Feature with the role "server", the Actor has the server role for this data. This means that the Actor must provide the corresponding data set of the Specialization as part of its Features. Further details are described in the sub-section "Server data - Resources".

3.2.4.2 Server data - Resources

3.2.4.2.1 Overview

Behind the entityType "HVACRoom", the Actor HVAC Room SHALL offer the Feature Types and Functions given in the table below.

Feature Type	**Scenario: M/R/O**	**Function**	**Possible operations**
DeviceClassification	3: M	deviceClassificationUserData	read (M)

Table 11: Feature Types and Functions used within this Use Case by the Actor HVAC Room

For each of these Feature Types, the following rule applies: There SHALL be at maximum one Feature with the Feature Type in the Entity.

Note: As a consequence of the previous rule, an implementation may need to have Feature data from different Scenarios/Specializations or even Use Cases in a given Feature.

The Scenario number shows in which Scenarios the HVAC Room acts as server and which Feature Types and Functions are relevant in each Scenario.

A detailed definition of the Elements and values that shall be supported in each Function is given in the following sub-sections.

Note: If in the table above "partial" read is not mentioned or is only optional, it still might be mandatory to support partial notifications. The details of "partial" support are described within the Scenario sections.

Note: The presence indications stated above are meant relative to the ones of the according Scenario stated in Table 1. I.e., if a Scenario is optional ("O") and a Feature Type is mandatory ("M"), the Feature Type need only be supported if the Scenario is supported, too.

Note: Further Features MAY be implemented on the same Entities; also, further Functions MAY be implemented in the used Entities.

3.2.4.2.2 Feature Type "DeviceClassification"

3.2.4.2.2.1 Function "deviceClassificationUserData"

Scenario [{…}]: M/R/O [\W][\C]	Element	Value	[High-Level Mapping] Element and value rules
3: M	DeviceClassification. deviceClassificationUserData.		
3: M	userLabel		[VHAN-003] The string-length SHOULD NOT be longer than 256 characters. If it is longer, the sender SHALL consider the possibility that the receiver will shorten the string to 256 characters.

Table 12: Content of Function "deviceClassificationUserData" at Actor HVAC Room

3.2.4.3 Client data - Specializations

As this Actor has only server functionality, no Specializations are described within this section.

3.3 Pre-Scenario communication

3.3.1 General information

The Pre-Scenario communication is needed if a client does not know the corresponding addresses on the server or if the required subscriptions or bindings are not active. In this case certain general communication mechanisms SHALL be used within SPINE:

a) Detailed discovery: allows to discover resource addresses.
b) Binding: allows to bind to resource address, which is frequently necessary to obtain write privileges.
c) Subscription: allows to subscribe to resource addresses, which is necessary to receive unsolicited notifications if a resource changes during runtime.

It is possible to combine those steps for multiple Scenarios or also multiple Use Cases:

- E.g. if multiple Scenarios in multiple Use Cases use the same Feature, only one subscription needs to occur.
- E.g. a complete detailed discovery or a subscription to the NodeManagement Feature needs to occur only once for all Use Cases.

Depending on which Entity, Feature and Functions are used within a Scenario the payload of the corresponding messages may slightly differ, but the basic principles and messages used stay the same.

The subsequent messages SHALL be exchanged for those parts that have not already been performed since the current connection is established or if those parts are outdated for another reason (e.g. if the detailed discovery is needed, but the bindings and subscriptions are still active from a previous connection only the detailed discovery messages need to be exchanged). If all Pre-Scenario communication parts are up to date, this section MAY be skipped, and the implementation can proceed as described in the corresponding "Scenario communication" sections.

After the connection is re-established (e.g. due to a power loss or a firmware update) the Pre-Scenario communication SHALL be performed as well. There may be circumstances where messages from the Pre-Scenario communication may be exchanged again.

Often the necessary messages of different Scenarios can be combined, so that only one single message is needed instead of multiple messages for the different Scenarios. This also is the case for the Pre-Scenario communication. In most cases only one "read" operation on the detailed discovery is necessary, as well as only one subscription request or binding request is needed for each Feature. Often multiple Scenarios within a Use Case access the same Feature, so only one subscription or binding is necessary.

3.3.2 Detailed discovery

For the functionality where a client already has current detailed discovery information (i.e. independent of this Use Case or any Scenario of it) the remainder of this section SHOULD be skipped.

Otherwise, the following procedure SHALL be performed in the given order:

1. If a client is not subscribed to the primary NodeManagement instance, the client SHALL acquire a subscription according to the figure provided within this sub-section.
2. A client SHALL read the detailed discovery information according to the figure provided within this sub-section. It SHALL keep the received information as far as it concerns mandatory and supported optional Entity Types, Feature Types and Functions of this Use Case that are needed by the client. This means that a client may choose how to store the necessary information. E.g. a client Actor can store the information how to address the necessary Features of the implemented Scenarios but may discard the Entity information.
3. If and as long as a client has a subscription to the detailed discovery information of an Actor and receives proper notifications, it SHALL consider (i.e. integrate into the kept detailed discovery information) the received information as far as it concerns mandatory and supported optional Entity Types, Feature Types and Functions of this Use Case.

Figure 8: Pre-Scenario communication - Detailed discovery sequence diagram

If the "nodeManagementDetailedDiscoveryData read" fails, the client SHOULD retry to read the detailed discovery information until the "nodeManagementDetailedDiscoveryData reply" message was received successfully.

If all functionality is present at all times: The "nodeManagementDetailedDiscoveryData reply" message contains at least the mandatory Entities and Features given in the "Actor [...] overview" diagrams as well as the used Functions and their "possible operations" described in section 3.2 and its sub-sections.

If functionality is added or removed dynamically: The "nodeManagementDetailedDiscoveryData reply" message does not need to contain all mandatory Entities and Features given in the "Actor [...] overview" diagrams as well as all needed Functions and their "possible operations" described in section 3.2 and its sub-sections. However, as soon as the functionality is available it will be announced via a "nodeManagementDetailedDiscoveryData notify" message.

For the nodeManagementDetailedDiscoveryData read Function it is recommended to use a partial read with separated Selectors that may use one of the following Elements:

- entityType
- featureType

Note: Even with the usage of Selectors Features and Entities that are not relevant for this Use Case may be discovered. However, only Features and Entities that fulfil the hierarchical order as described within the Actors' sections shall be considered for this Use Case.

A "partial" notify SHALL be supported without using Selectors and Elements. Partial "delete" notify SHOULD also be supported with separated Selectors that may use one of the following Elements:

- entityAddress
- featureAddress

3.3.3 Binding

If binding is required by a Scenario that uses Features with writeable or changeable data, the server SHALL support binding for the respective Features. Before a write on a Function of a Feature occurs, the client SHALL create a binding to the corresponding Feature. For this the nodeManagementBindingRequestCall Function is used as shown in the following sequence diagram:

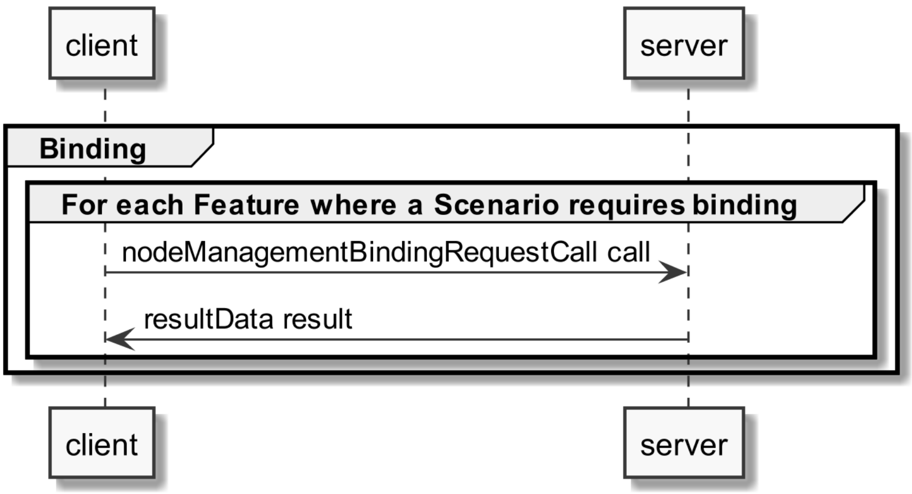

Figure 9: Pre-Scenario communication - Binding sequence diagram

If functionality is added or removed dynamically, binding may not be possible at all times on the required Functions. A client SHALL retry to create a binding again when receiving according updated detailed discovery information.

3.3.4 Subscription

A server SHALL support subscription for all Features that contain readable data that may change during runtime. The client SHALL create a subscription for all Features that the client wants to read. For this the nodeManagementSubscriptionRequestCall Function is used as shown in the following sequence diagram:

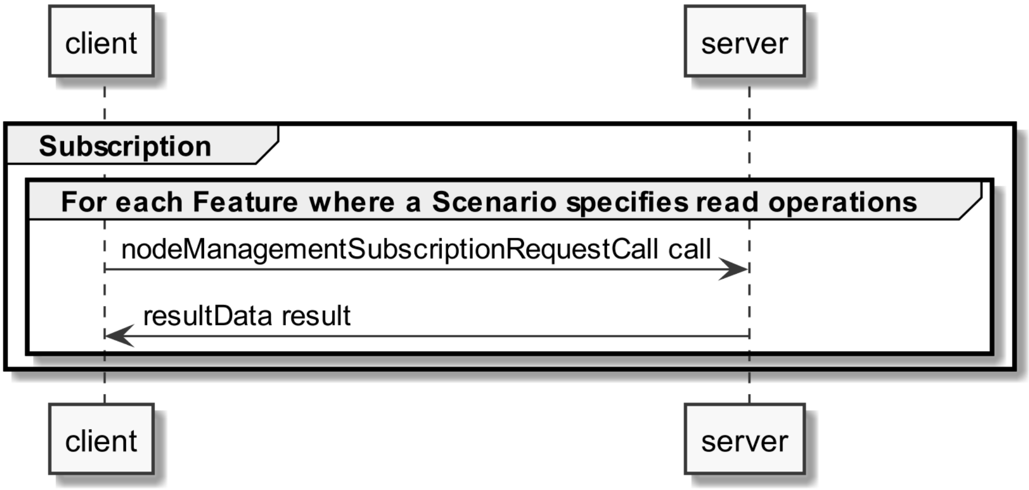

Figure 10: Pre-Scenario communication - Subscription sequence diagram

If the subscription request fails (e.g. because it is not supported by the server or the maximum number of possible subscriptions is reached), the client SHOULD read the data periodically (so-called "polling").

If functionality is added or removed dynamically, subscription may not be possible at all times on the required Functions. A client SHALL retry its subscription procedure again when receiving according updated detailed discovery information.

3.3.5 Dynamic behaviour

In case Entities or Features are removed, a nodeManagementDetailedDiscoveryData "notify" is transmitted that informs about the deleted Entities and Features. All existing binding or subscription entries on the deleted Features SHALL be deleted by each device.

In case Entities or Features are added the Pre-Scenario communication starts with transmitting a nodeManagementDetailedDiscoveryData "notify" that contains the added Entities and Features.

3.4 Scenarios

3.4.1 Scenario 1 - Visualize heating circuit name

3.4.1.1 Pre-Scenario communication

1. **Detailed discovery**: Actors that act as client within this Scenario, need to know the addresses of the server Features used in the Initial Scenario communication. If the address of a particular server Feature is not known, the detailed discovery must be used, as described in section 3.3.2.
2. **Binding**: Binding SHOULD NOT be used for this Scenario.
3. **Subscription**: Actors SHALL create a subscription for each server Feature that is relevant for the corresponding Actor within this Scenario, as described in section 3.3.4.

The Initial Scenario communication SHALL start at the latest when the required resources on an Actor are known and the necessary binding and subscription procedures have been finished. However, as soon as the address of a required resource is known, the Initial Scenario communication for this resource MAY start already, even if the addresses of other required resources are not known yet.

If required resources are removed and added again, they are re-discovered, and the Initial Scenario communication is triggered again for those resources.

3.4.1.2 Initial Scenario communication

Each time a (re-)connection is established, even if the Pre-Scenario communication phase is skipped, the messages shown in the following sequence diagram SHALL be exchanged, as the corresponding resources may have changed in the meantime:

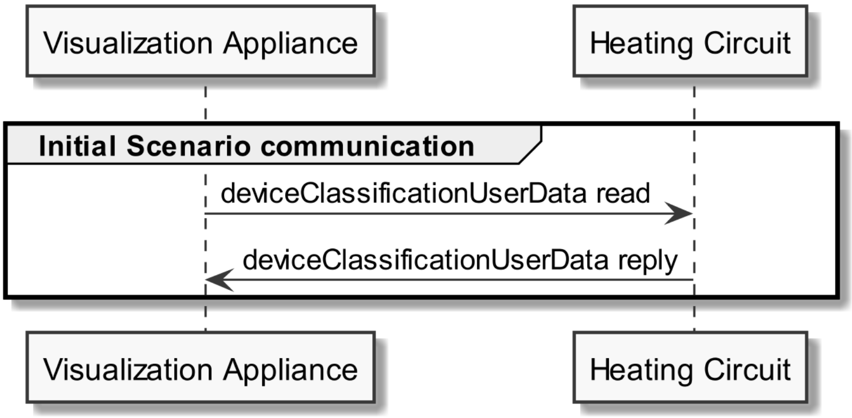

Figure 11: Scenario 1 - Initial Scenario communication sequence diagram

The following table shows where the required content of the messages from the sequence diagram is described:

Message name from sequence diagram	**Content description in table**	**Scenario number in table**
deviceClassificationUserData reply	Table 8	1

Table 13: Initial Scenario communication content references for Scenario 1

Note: Within the Initial Scenario communication the content required by this Scenario MAY not be provided completely, but later during Runtime Scenario communication.

3.4.1.3 Runtime Scenario communication

Based on the Initial Scenario communication, the Runtime Scenario communication provides updates during runtime.

If one of the referenced server Functions' data change, the server SHALL submit the change as shown in the following figure:

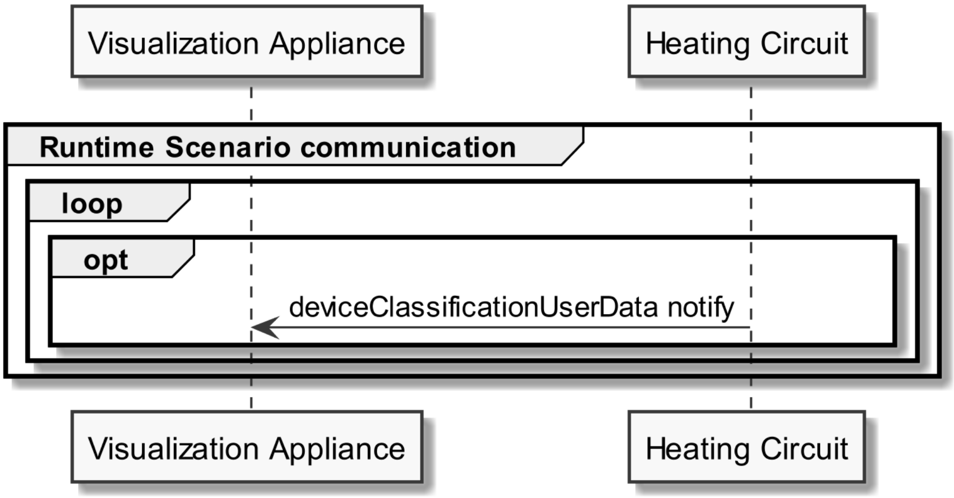

Figure 12: Scenario 1 - Runtime Scenario communication sequence diagram

Note: To interpret partial notification messages correctly, the information obtained during the Initial Scenario communication phase is required.

Note: A read operation ("polling") on all Functions is possible at any time, e.g. if a notification could not be evaluated.

The following table shows where the required content of the messages of the sequence diagram is described:

Message name from sequence diagram	Content description in table	Scenario number in table
deviceClassificationUserData notify	Table 8	1

Table 14: Runtime Scenario communication content references for Scenario 1

3.4.1.4 Additional information

None.

3.4.2 Scenario 2 - Visualize heating zone name

3.4.2.1 Pre-Scenario communication

1. **Detailed discovery**: Actors that act as client within this Scenario, need to know the addresses of the server Features used in the Initial Scenario communication. If the address of a particular server Feature is not known, the detailed discovery must be used, as described in section 3.3.2.
2. **Binding**: Binding SHOULD NOT be used for this Scenario.
3. **Subscription**: Actors SHALL create a subscription for each server Feature that is relevant for the corresponding Actor within this Scenario, as described in section 3.3.4.

The Initial Scenario communication SHALL start at the latest when the required resources on an Actor are known and the necessary binding and subscription procedures have been finished. However, as soon as the address of a required resource is known, the Initial Scenario communication for this resource MAY start already, even if the addresses of other required resources are not known yet.

If required resources are removed and added again, they are re-discovered, and the Initial Scenario communication is triggered again for those resources.

3.4.2.2 Initial Scenario communication

Each time a (re-)connection is established, even if the Pre-Scenario communication phase is skipped, the messages shown in the following sequence diagram SHALL be exchanged, as the corresponding resources may have changed in the meantime:

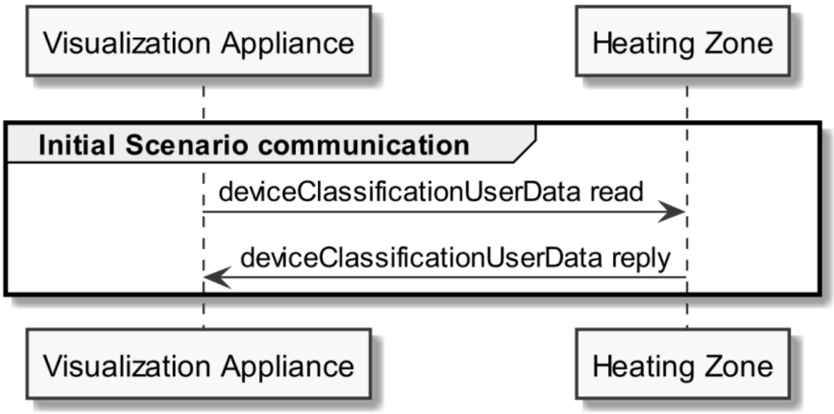

Figure 13: Scenario 2 - Initial Scenario communication sequence diagram

The following table shows where the required content of the messages from the sequence diagram is described:

Message name from sequence diagram	Content description in table	Scenario number in table
deviceClassificationUserData reply	Table 10	2

Table 15: Initial Scenario communication content references for Scenario 2

Note: Within the Initial Scenario communication, the content required by this Scenario MAY not be provided completely, but later during Runtime Scenario communication.

3.4.2.3 Runtime Scenario communication

Based on the Initial Scenario communication, the Runtime Scenario communication provides updates during runtime.

If one of the referenced server Functions' data change, the server SHALL submit the change as shown in the following figure:

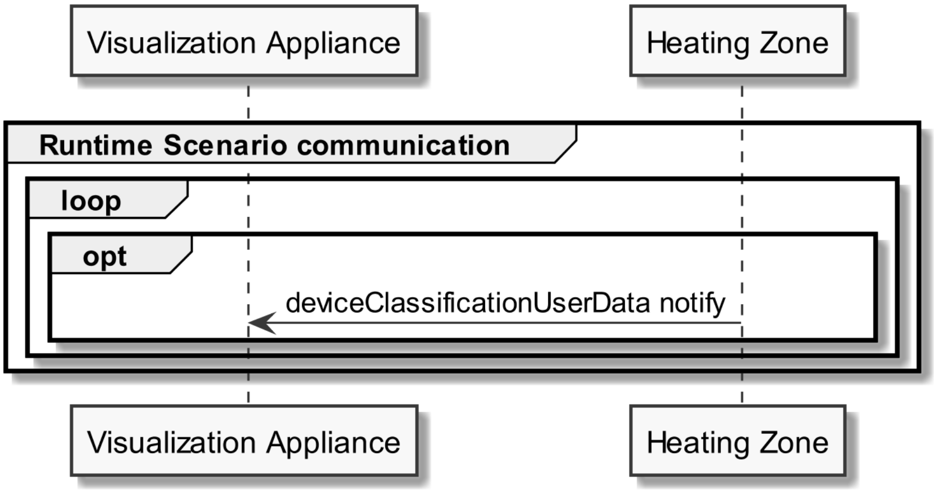

Figure 14: Scenario 2 - Runtime Scenario communication sequence diagram

Note: To interpret partial notification messages correctly, the information obtained during the Initial Scenario communication phase is required.

Note: A read operation ("polling") on all Functions is possible at any time, e.g. if a notification could not be evaluated.

The following table shows where the required content of the messages of the sequence diagram is described:

Message name from sequence diagram	**Content description in table**	**Scenario number in table**
deviceClassificationUserData notify	Table 10	2

Table 16: Runtime Scenario communication content references for Scenario 2

3.4.2.4 Additional information

None.

3.4.3 Scenario 3 - Visualize heating room name

3.4.3.1 Pre-Scenario communication

1. **Detailed discovery**: Actors that act as client within this Scenario, need to know the addresses of the server Features used in the Initial Scenario communication. If the address of a particular server Feature is not known, the detailed discovery must be used, as described in section 3.3.2.
2. **Binding**: Binding SHOULD NOT be used for this Scenario.
3. **Subscription**: Actors SHALL create a subscription for each server Feature that is relevant for the corresponding Actor within this Scenario, as described in section 3.3.4.

The Initial Scenario communication SHALL start at the latest when the required resources on an Actor are known and the necessary binding and subscription procedures have been finished. However, as soon as the address of a required resource is known, the Initial Scenario communication for this resource MAY start already, even if the addresses of other required resources are not known yet.

If required resources are removed and added again, they are re-discovered, and the Initial Scenario communication is triggered again for those resources.

3.4.3.2 Initial Scenario communication

Each time a (re-)connection is established, even if the Pre-Scenario communication phase is skipped, the messages shown in the following sequence diagram SHALL be exchanged, as the corresponding resources may have changed in the meantime:

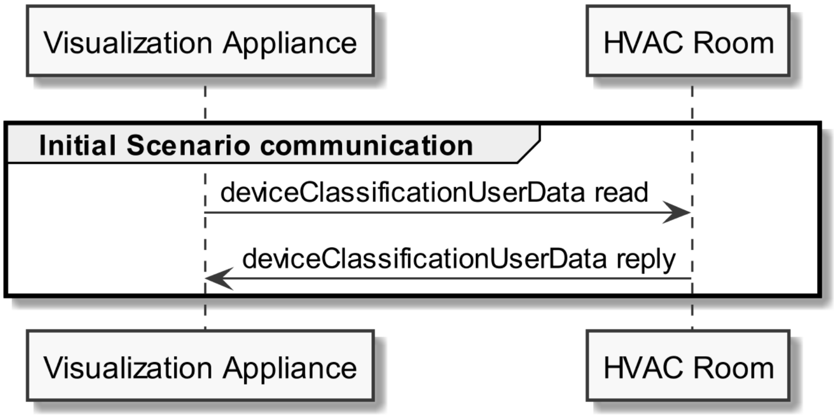

Figure 15: Scenario 3 - Initial Scenario communication sequence diagram

The following table shows where the required content of the messages from the sequence diagram is described:

Message name from sequence diagram	Content description in table	Scenario number in table
deviceClassificationUserData reply	Table 12	3

Table 17: Initial Scenario communication content references for Scenario 3

Note: Within the Initial Scenario communication, the content required by this Scenario MAY not be provided completely, but later during Runtime Scenario communication.

3.4.3.3 Runtime Scenario communication

Based on the Initial Scenario communication, the Runtime Scenario communication provides updates during runtime.

If one of the referenced server Functions' data change, the server SHALL submit the change as shown in the following figure:

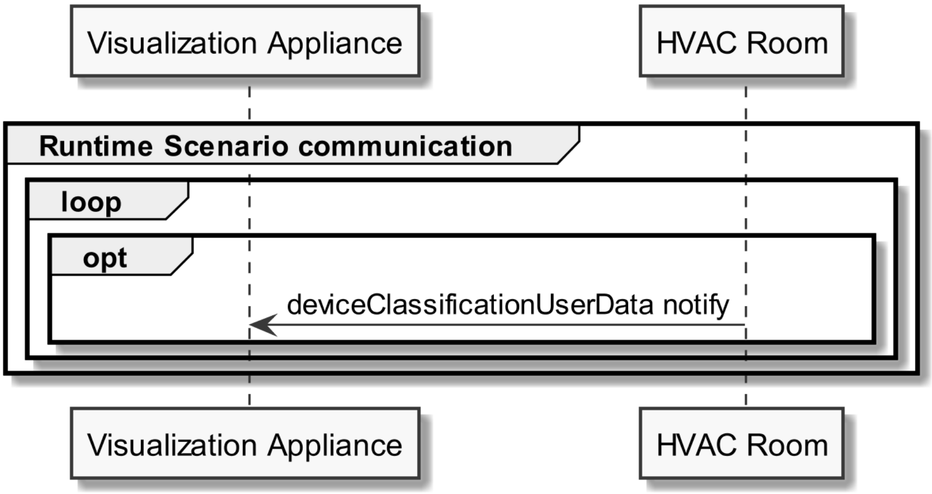

Figure 16: Scenario 3 - Runtime Scenario communication sequence diagram

Note: To interpret partial notification messages correctly, the information obtained during the Initial Scenario communication phase is required.

Note: A read operation ("polling") on all Functions is possible at any time, e.g. if a notification could not be evaluated.

The following table shows where the required content of the messages of the sequence diagram is described:

Message name from sequence diagram	**Content description in table**	**Scenario number in table**
deviceClassificationUserData notify	Table 12	3

Table 18: Runtime Scenario communication content references for Scenario 3

3.4.3.4 Additional information

None.

